

Requirements Engineering

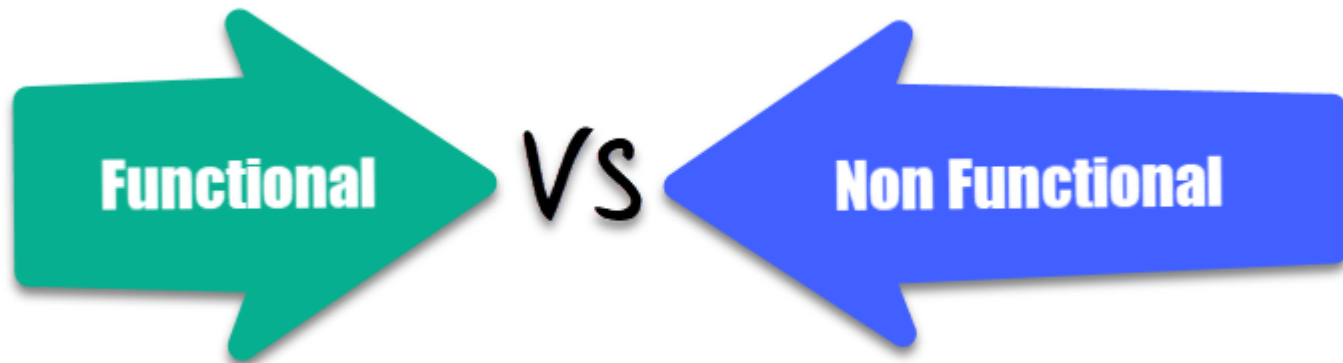
Requirements Definition and Specification

Requirements definition

- Abstract description of the **services** which the system should provide and the **constraints** under which the system must operate;
- Should be written in such a way that it is understandable by customers without knowledge of specialized notations

Requirements specification – Structured document which sets out the system service in detail; Should be precise.

Requirements Definition



Functional Requirements

Functional requirements (**WHAT**)

- Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks.
- So, it's important to make them clear both for the development team and the stakeholders.
- Requirements should not contain design and implementation details (**HOW**)

Functional Requirements (contd.)

Functional Requirements should include the following:

Details of operations conducted in every screen

Data handling logic should be entered into the system

It should have descriptions of system reports or other outputs

Complete information about the workflows performed by the system

It should clearly define who will be allowed to create/modify/delete the data in the system

How the system will fulfill applicable regulatory and compliance needs should be captured in the functional document

Functional Requirements (contd.)

Here's an example list of functional requirements for a user interacting with an Automated Teller Machine (ATM):

- The system shall prevent further interaction if it's out of cash or is unable to communicate with the financial institution.
- The system shall validate that the inserted card is valid for financial transactions on this ATM.
- The system shall validate that the PIN number entered by the user is correct.
- The system shall dispense the requested amount of money, if it is available, and debit the user's account by the same amount.
- The system shall notify the user if the transaction could not be completed. In that case, no money shall be taken from the user's account.

Non Functional Requirements

Nonfunctional requirements:

- Describe the general characteristics of a system.
- They are also known as ***quality attributes***.
- Failing to meet non-functional requirements can result in systems that fail to satisfy user needs.

Non Functional Requirements (contd.)

Non-Functional Requirements should include the following (but are not limited to) :

- **Performance.** How fast does the system return results?
- **Scalability.** How much will this performance change with higher workloads?
- **Portability.** Which hardware, operating systems, browsers, and their versions does the software run on?
- **Compatibility.** Does it conflict with other applications and processes within these environments?
- **Reliability.** How often does the system experience critical failures? and
- **Availability.** How much time is it available to users against downtimes?
- **Security.** How are the system and its data protected against attacks?
- **Usability.** How easy is it for a customer to use the system?

Non Functional Requirements (contd.)

Here are examples list of non-functional requirements:

If Employees try to update their salary information, such attempt should be reported to the security administrator.

Every unsuccessful attempt by a user to access an item of data shall be recorded on an audit trail.

A website should be capable enough to handle 20 million users without affecting its performance

The software should be portable. So moving from one OS to other OS does not create any problem.

Privacy of information, the export of restricted technologies, intellectual property rights, etc. should be audited.

A Decision Support System for Office Space Management

CASE STUDY

Case Study Aim

In any sizeable organization, managing the massive amount resources can become and expensive and a time consuming task.

One of these many resources is the building(s) which must be adequate to both house and allow efficient operation of all entities, a entity being a member of staff or a facility.

Allocating enough building space for every required entity in a organization can become problematic with the many restrictions and limited space.

Many institutes have planning and layout guidelines which must be followed, plus each room may also have special requirements. Being able to plan a suitable layout can take considerable time and any future changes will consume yet more time and money.

Making a optimized floor plan that wastes the minimal amount of space, can accommodate future changes in both staff and facilities and meets all restrictions is the problem.

Case Study Functional Requirements

1. Creation and editing of 2D floor plans for one building floor using straight lines only
2. Creation and editing of entities
3. Creation and editing of constraints
4. A user interface encompassing the floor plan, entities, and constraints
5. A print option or a exportable format suitable for printing which can produce a document suitable for use as a floor layout diagram
6. The ability to manually edit entities assigned rooms
7. The ability to manually assign room names
8. The ability to save and load Space Allocation data
9. Automatically edit rooms and assign entities based upon the constraints, entities, and the floor plan

Case Study Non Functional Requirements

1. Runs on Microsoft Windows XP
2. Allow quick, precise, understandable, and easily accessible editing of in-program data
3. Provide an accessible and usable User Interface to someone who is has basic computer knowledge
4. Has a data representation that allows AI algorithms to propose an optimized floor plan
5. To provide extensive documentation for both a user and a systems developer
6. Have a clean object oriented design allowing good maintainability

Requirements Engineering

(1) Requirements elicitation

process through which the customers, buyers, or users of a software system discover, reveal, articulate, and understand their requirements

(2) Requirements analysis and negotiation

process of reasoning about the requirements that have been elicited; examining requirements for **conflicts** or **inconsistencies**, **combining related requirements**, and identifying **missing** requirements

(3) Requirements specification

process of recording the requirements in one or more **forms**, including **natural** language and **formal**, **symbolic**, or **graphical** representations;

Also, the product – document produced by the process

Requirements Engineering

(1) REQUIREMENTS ELICITATION

Requirements Elicitation

In requirements engineering, **requirements elicitation** is the practice of researching and discovering the requirements of a system from users, customers, and other stakeholders.

The practice is also sometimes referred to as "**requirement gathering**".

It's doubtful that any single elicitation technique will always work for you.

It is generally accepted that an individual requirements elicitation technique or approach cannot possibly be suitable for all projects.

So how do you decide which will give you the most bang for your buck (limited time), and why?

- Your organization's makeup,
- political climate,
- the nature of your project,
- your personal strengths and preferences

will have a lot to do with which methods work best for you.

Requirement Elicitation Process

Step 1: Choose the optimal elicitation approach

Step 2: Elicitation preparation

Step 3: Conduct the elicitation

Step 4: Analyze elicitation results

Requirements Elicitation Techniques

There are a myriad of requirements elicitation methods:



but there are many more methods out there...

The Top 5 Elicitation Techniques



#1 Prototyping

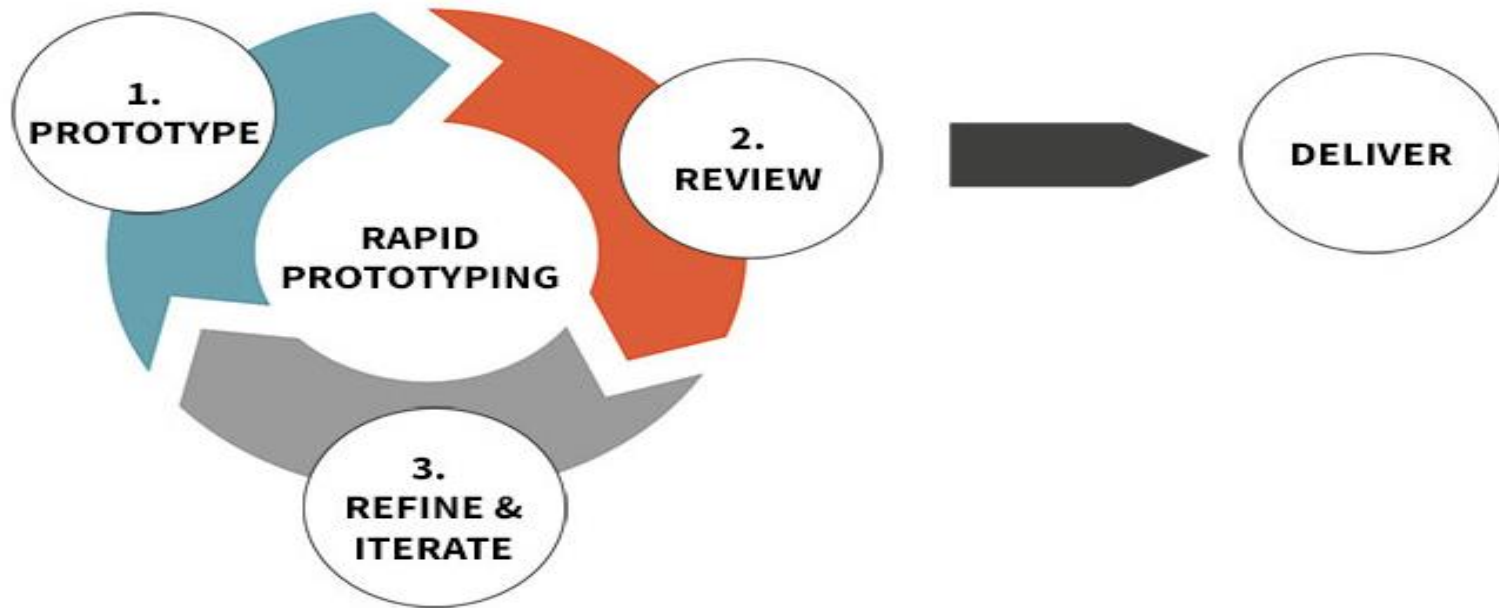
#2 Workshops

#3 Interviews

#4 Surveys

#5 Observation

Prototyping



Benefit: You can make sure that what you're designing is really what people need while you still have time to change it.

Prototyping Example

MLCAT - Podcasting Lectures onto Mobies

select an icon
MLCAT

Create/Edit Session

Course Title:

Description:

Date:

Lecturer:

Venue:

Upload Files:

Upload files takes you to another screen which allows files to be added.

Requirements Workshops

In a requirements workshop, you ask everyone to sit down and hammer out the requirements with you.

“A requirements workshop:

- is a highly productive focused event
- attended by carefully selected key stakeholders and subject matter experts for a short,
- intensive period (typically one or a few days).”

Benefit: You can get your basic requirements done in a hurry. Also, everyone you invite can become invested in the project.

Requirements Workshops



Requirements Workshops

For your workshop to be successful, you will need to:

Select the right participants:

- Think about this carefully before inviting your group. (Do not involve too many participants → slow down the workshop process)
- Conversely, collecting input from too few participants can lead to overlooking requirements.”

Get everyone on the same page regarding the purpose of the workshop ahead of time (defining scope, unearthing business requirements, etc.)

Conduct the workshop like an interview, with open-ended questions presented to the room.

Document everything. Get a recorder or get someone besides you (or whomever will be busy facilitating) to write everything down.

Interviews

Interviews help you dig through your users' knowledge base, so you can understand what they understand and think.

One writer notes, "Interviews provide an efficient way to collect large amounts of in-depth data quickly,"

Benefit: By exploring someone's knowledge and needs in-depth, one-on-one, you ensure you understand the real, not just the perceived, need.

Interviews



Interviews (contd.)

For your interview to be effective, you must decide how structured or unstructured you want your interview to be.

Structured Interviews:

- These are interviews that strictly adhere to the use of an interview protocol to guide the researcher.
- It is a more rigid interview style, in that only the questions on the interview protocol are asked.
- As a result, there are not a lot of opportunities to probe and further explore topics that participants bring up when answering the interview questions.
- **This method can be advantageous when researchers have a comprehensive list of interview questions**

Interviews (contd.)

Semi structured interviews:

- These are interviews that use an interview protocol to help guide the researcher through the interview process.
- It does maintain some structure (hence the name semi structured),
- But it also provides the researcher with the ability to probe the participant for additional details.
- If you decide to choose this interview method, understand that it offers a great deal of flexibility for you as a researcher.
- You do not have to worry about needing to conduct several rounds of interviews **because your interview protocol will keep you focused on gathering all the information** that you need to answer your research question.

Interviews (contd.)

Unstructured interviews:

- These are interviews that take place with few, if any, interview questions.
- They often progress in the manner a normal conversation would, however it concerns the research topic under review.
- It is a relatively formless interview style that researchers use to establish rapport and comfort with the participant, and is extremely helpful when researchers are discussing sensitive topics.
- If you select this interview style, just keep in mind that you **may have to conduct several rounds of interviews** with your participants in order to gather all the information you need.
- Since you do not use a standard interview protocol, sometimes participant's narratives maneuver the conversation away from other aspects of the research topic you want to explore;

Surveys /Questionnaires

Questionnaires take into consideration data to be evoked from **numerous individuals**.

The ideal approach to this technique is by making a basic Google Form and offering it to the correct individuals, and whenever required, determining a due date.

You have to know what you are attempting to accomplish precisely with the study, and the questions must not to be uncertain.

Misunderstanding of inquiries can prompt useless and pointless answers.

The format for Questionnaires

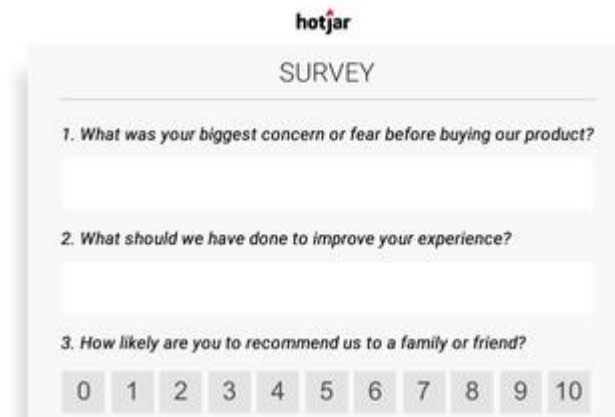
Fixed Format:

- Fixed format surveys consist of questions that need a variety of predefined responses from people.
- Respondents have to choose an answer from a series of answers provided.
- A reply from this format of the questionnaire is a lot simpler to interpret.
- In any case, then again, it is increasingly latent; respondents can't give their answers or opinion other than presented in the survey.

Free Format:

- Free format surveys will enable users to answer openly for each inquiry.
- A question is proposed, and the respondent enters the appropriate response in the space given after the query.

Free Format



A Hotjar survey widget is displayed. At the top is the Hotjar logo. Below it is the word "SURVEY" in all caps. The survey contains three questions. The first two are open-ended text questions, and the third is a Likert scale question.

hotjar

SURVEY

1. What was your biggest concern or fear before buying our product?

2. What should we have done to improve your experience?

3. How likely are you to recommend us to a family or friend?

0 1 2 3 4 5 6 7 8 9 10

Observation

Observation is primarily useful for capturing what's already in existence and enables several other types of requirements tools.

The analyst can document what he/she observes through numerous types of diagramming and business process models.

“The effectiveness of observation . . . can vary as users have a tendency to adjust the way they perform tasks when knowingly being watched.”

Make sure the people you observe know that you are not there to judge what they do, but to make their work easier in the long run.

Ask them what they like and don't like about it, and about any workarounds they've created on their own.

Benefit: You can figure out exactly where users are at the start of your project, and you can use your strengths to document it.

Observation



Summary of Elicitation Techniques

Technique	Good for	Kind of data	Plus	Minus
Questionnaires	Answering specific questions	Quantitative and qualitative data	Can reach many people with low resource	The design is crucial. Response rate may be low. Responses may not be what you want
Interviews	Exploring issues	Some quantitative but mostly qualitative data	Interviewer can guide interviewee. Encourages contact between developers and users	Time consuming. Artificial environment may intimidate interviewee
Focus groups and workshops	Collecting multiple viewpoints	Some quantitative but mostly qualitative data	Highlights areas of consensus and conflict. Encourages contact between developers and users	Possibility of dominant characters
Naturalistic observation	Understanding context of user activity	Qualitative	Observing actual work gives insight that other techniques cannot give	Very time consuming. Huge amounts of data

Requirement Elicitation Process

Step 1: Choose the optimal elicitation approach

Step 2: Elicitation preparation

Step 3: Conduct the elicitation

Step 4: Analyze elicitation results

Elicitation Preparation

You should have established:

An objective for elicitation activities

The specific participants

The resources or other supporting materials needed during the elicitation activities

A predetermined set of questions and how stakeholder responses are to be recorded

An agenda with definitive start and end times

Requirement Elicitation Process

Step 1: Choose the optimal elicitation approach

Step 2: Elicitation preparation

Step 3: Conduct the elicitation

Step 4: Analyze elicitation results

Conduct the Elicitation

Elicitation activities have four stages:

1. Introduction:

- You set the stage by introducing the purpose and goals of the elicitation activity: establish the tone of the interaction, discussion of activities, timing, etc.

2. Body:

- Depending on which tool or technique you chose, the body will differ.
- For example, if you're interviewing a stakeholder, you'll likely launch into your questions or if you're conducting a workshop, you'll transition the group into the scheduled activity.
- Questions may arise organically throughout the interaction.

Conduct the Elicitation (contd.)

3. Close:

- The transition to elicitation activity closure can be a smooth one if you've kept an eye on the time in relation to the flow of the activity.
- Ask closing questions, e.g., "Do you have any questions", "Is there any information you think is important about the product that we didn't discuss?", etc.
- Definitely ask if the stakeholder is open to follow up questions, new ones may surface as you analyze the elicitation results.

4. Follow-up:

- **This step isn't always needed**, but can prove to be extremely useful.
- Follow-ups provide you with the opportunity to ask clarifying questions and ensure that the information you did record is accurate.

Conduct the elicitation (contd.)

Once all of your elicitation activities are completed, you've gathered all of the data/information into some sort of repository

- this can be digital
- handwritten notes,
- drawings, etc.

Your next step is to analyze everything you've collected.

Requirement Elicitation Process

Step 1: Choose the optimal elicitation approach

Step 2: Elicitation preparation

Step 3: Conduct the elicitation

Step 4: Analyze elicitation results

Analyze Elicitation Results

Analysis can be as straightforward as reading through your notes and other documents then summarizing them.

Identifying keywords and ideas that were derived from stakeholder elicitation activities.

There are tools to support the analysis of requirements.

Difficulties of Requirements Elicitation

(1) Articulation problems

Users: may **not** be **aware** of their needs, **unable** to articulate them appropriately or **afraid** to articulate them

Developers: may not really be **listening** to the users; may fail to **understand**, appreciate, or relate to the users; tend to overrule or **dominate** users

(2) Communication barriers:

Users and developers have different professional vocabularies

(3) Knowledge and cognitive limitations:

Requirements elicitor must have adequate domain knowledge

People tend to state the problem in terms of the **avored solution**

Difficulties of Requirements Elicitation (contd.)

(4) Human behavior issues:

Expectation or fear that installation of software will necessitate all kinds of changes in behavior, including the potential **loss of jobs**.

(5) Technical issues:

Requirements change over time

Software and hardware **technologies** are **changing** rapidly

Requirements Engineering

(1) Requirements elicitation

process through which the customers, buyers, or users of a software system discover, reveal, articulate, and understand their requirements.

(2) Requirements Analysis and negotiation

process of examining the requirements that have been elicited; Prioritizing the elicited requirements; Classify the requirements as discussed in lecture ;

(3) Requirements specification

process of recording the requirements in one or more forms, including natural language and formal, symbolic, or graphical representations;

Also, the product – document produced by the process

Requirements Engineering

(2) REQUIREMENTS ANALYSIS

Requirements Analysis

Requirements analysis, is the process of determining user expectations for a new or modified product.

These features, called requirements, must be quantifiable, relevant and detailed.

In software engineering, such requirements are often called functional specifications.

Requirements analysis is an important aspect of project management.

Requirements Analysis (contd.)

For Analyzing Requirements take the following steps:

- Make Requirements **S.M.A.R.T**
- **MoSCoW** the Requirements
- Classify the Requirements

Make them S.M.A.R.T.

S



SPECIFIC

M



MEASURABLE

A



ACHIEVABLE

R



RELEVANT

T



TIME-BASED

Specific Requirements

Without ambiguity, using consistent terminology, simple and at the appropriate level of detail.

Let's consider this requirement: ***The new system shall be able to manage project schedule.***

Is the requirement specific? The Answer is NO.

What is meant by the verb Manage? Does it mean Create – Retrieve – Update – Delete or something more?

Measurable Requirements

Is it possible to put a number to the requirement?

This is especially true for non-functional requirements.

Let's consider the requirement: *The system shall have great usability.*

How do we measure great usability?

One way to measure is through the Success rate/ completion rate, which is the percentage of users who were able to successfully complete the tasks.

Achievable Requirements

By an achievable requirement means that it could be reasonably accomplished under the given conditions and within the bounds of human knowledge.

For example: *"The system shall be 100% reliable and 100% available"*. OR *"The system shall have a minimum response to a query of 1 second irrespective of system load"*.

The consequence of attempting to meet these requirements is that the system will never be accepted or prohibitively expensive or both .

Relevant Requirements

Means the requirements are realistic, given the resources.

Aspects to consider include:

- Do we have the required staffing?
- Do we have the skill?
- Do we have access to the development infrastructure needed?
- Do we have access to the run-time infrastructure needed?
- Do we have enough time in hand to implement the requirement?

Let's consider this requirement: *Let's implement Oracle Applications in next 1 month.*

Any ERP project takes significant amount of planning and preparation. There would not have been any ERP implementation which is less than 3 months duration.

Time-oriented Requirements

Answers the question, "when will it be done?"

Sometimes a task may only have an end point or due date.

Sometimes that end point or due date is the actual end of the task, or sometimes the end point of one task is the start point of another.

Sometimes a task has several milestones or check points to help you or others assess how well something is going before it is finished so that corrections or modifications can be made as needed to make sure the end result meets expectations.



S

- **SPECIFIC**
- Define the goal as much as possible with no unclear language
- **Who** is involved, **WHAT** do I want to accomplish, **WHERE** will it be done, **WHY** am I doing this – reasons, purpose, **WHICH** constraints and/or requirements do I have?



M

- **MEASURABLE**
- Can you track the progress and measure the outcome?
- How much, how many, how will I know when my goal is accomplished?



A

- **ATTAINABLE/ACHIEVABLE**
- Is the goal reasonable enough to be accomplished? How so?
- Make sure the goal is not out of reach or below standard performance.



R

- **RELEVANT**
- Is the goal worthwhile and will it meet your needs?
- Is each goal consistent with the other goals you have established and fits with your immediate and long term plans?



T

- **TIMELY**
- Your objective should include a time limit. Ex: I will complete this step by month/day/year.
- It will establish a sense of urgency and prompt you to have better time management.

MoSCoW the Requirements

MoSCoW is a method for prioritizing requirements.

The requirements are prioritized to prevent them from becoming too expensive or unrealistic.

The main goal is to come up with requirements that add the most value for the company.

The project requirements are divided into one of the following categories:

- Must-Haves
- Should-Haves
- Could-Haves
- Would-Haves

MoSCoW the Requirements (contd.)

M	Must have requirement
S	Should have if at all possible
C	Could have but not critical
W	Would be good to have... <i>(Won't have the time to do it now, but maybe later)</i>

M – Must haves

They are a necessity for a workable product and there is no alternative.

Without meeting these requirements, the project fails and the product won't be use-able.

Examples:

The HR system “must” store employee leave history.

S – Should haves

These are additional and much desired requirements that have a high priority, but are not essential for a usable end product.

When they are met, they will only add to the value of the product.

Depending on the available time, you can always return to these requirements at a later time.

Examples:

The HR system “should” allow printing of leave letters.

C – Could haves

The ‘Could haves’ have a lower priority than the ‘Should haves’.

This option will only be included if there really is more than enough time to make it work.

This category is also referred to as ‘nice to have’; they’re more a wish than an absolute requirement.

Examples:

The HR system “could” send out notifications on pending leave dates.

W – Won't haves (and would haves)

These are about wishes for the future that are often impossible to realize or cost a lot of time.

'Would haves' are often followed upon at a later stage after the initial project is finished.

Examples:

The HR system “won’t” support remote access but may do so in the next release.

MoSCoW PRIORITISATION

MUST HAVE

- Non-Negotiables
- Minimum Viable Product (MVP)
- Can't deliver on target date without this
- Not legal without it
- Unsafe without it
- Without this project is not viable

ask the question, "what happens if this requirement is not met?" if the answer is "cancel the project" there is no point implementing a solution that does not meet the requirement.

SHOULD HAVE

- Important but not vital
- Maybe painful to leave out but the solution is still viable
- May need some kind of workaround

A Should Have may be differentiated from a Could Have by reviewing the degree of pain caused by it not being met, in terms of business value or number of people affected.

COULD HAVE

- Desirable but not as important as Should Have
- Only do if there is extra time and budget

WON'T HAVE

- Won't have this time around at all
- Out of budget
- Nice to have but has no real impact

Maximum 60% of total effort

Contingency

Maximum 80% of total effort | Business Case

Maximum 100% of total effort

Classify the Requirements

At this stage:

The requirements were gathered in the ***Requirements Elicitation Phase***

The requirements were analyzed in the ***Requirements Analysis and Negotiation Phase:***

- Examined against SMART Objectives
- Prioritized by MoSCoW
- Then classified into two categories :
 - Functional Requirements
 - Non-functional requirements

Requirements Engineering

(2) REQUIREMENTS SPECIFICATION

Requirements Engineering

(1) Requirements elicitation

process through which the customers, buyers, or users of a software system discover, reveal, articulate, and understand their requirements.

(2) Requirements Analysis and negotiation

process of examining the requirements that have been elicited; Prioritizing the elicited requirements; Classify the requirements as discussed in lecture 4;

(3) Requirements specification

process of recording the requirements in one or more forms, including natural language and formal, symbolic, or graphical representations;

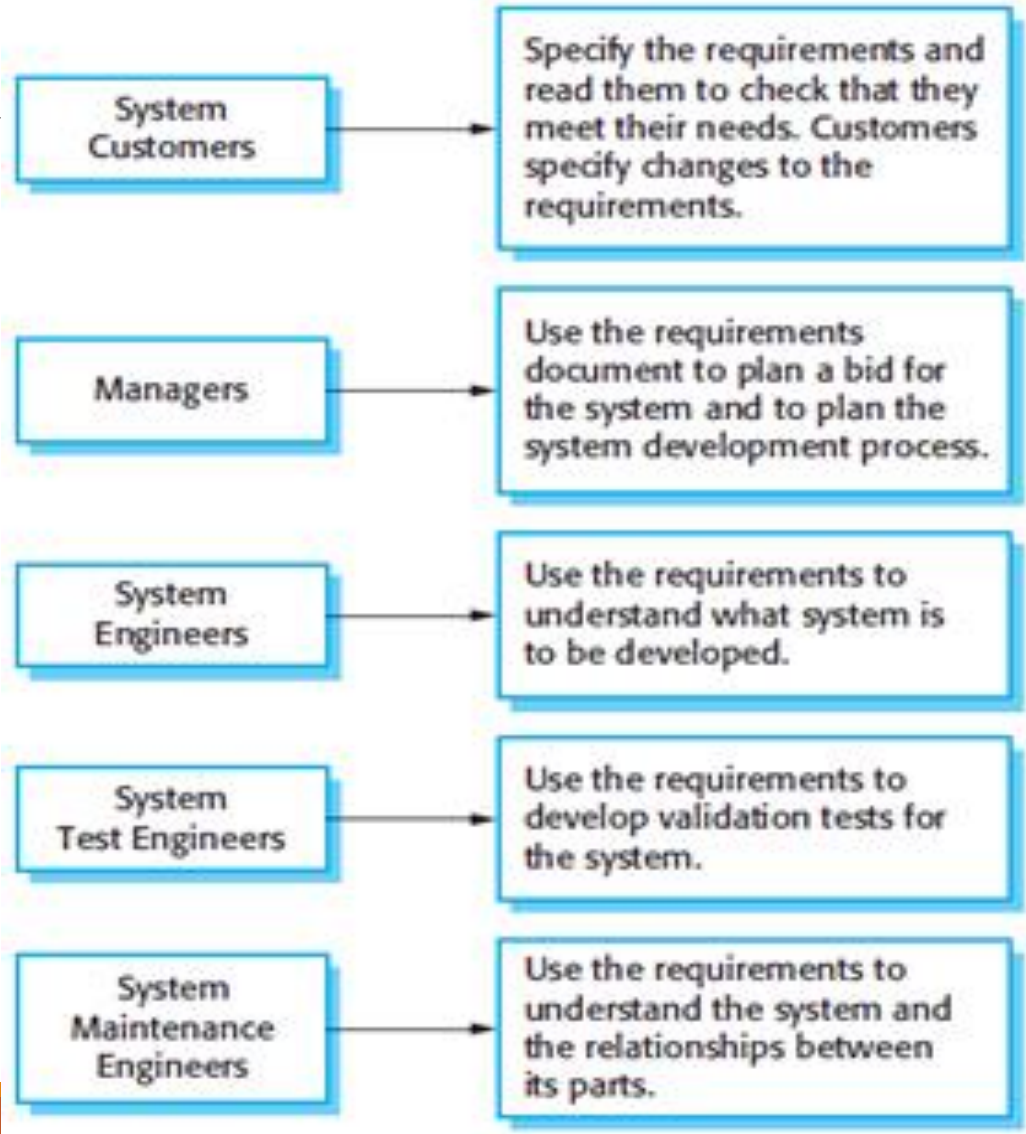
Also, the product – document produced by the process

Requirements Specification

Software requirements specification (SRS) is a **detailed description** of a software system to be developed with its functional and non-functional requirements.

The SRS is developed based the agreement between customer and contractors.

Users of a Requirements Document



Software Requirements Specification Template

(adapted from IEEE Standard 830-1998)

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience & Reading Suggestions
- 1.4 Project Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Features
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design & Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. System Features

- 3.x System Feature X
 - 3.x.1 Description and Priority
 - 3.x.2 Stimulus/Response Sequences
 - 3.x.3 Functional Requirements

4. External Interface Requirements

- 4.1 User Interfaces
- 4.2 Hardware Interfaces
- 4.3 Software Interfaces
- 4.4 Communications Interfaces

5. Other Nonfunctional Requirements

- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes

6. Other Requirements

Appendix A: Glossary

Appendix B: Analysis Models

Appendix C: Issues List

➤ IEEE Software Engineering standards: <http://standards.ieee.org/software>

Verification and Validation

Verification and Validation is the process of checking that a **software system meets specifications** and that it fulfills its intended purpose.

Verification: is a process of evaluating the intermediary work products of a software development lifecycle to check if we are on the right track of creating the final product.

Validation: is the process of evaluating the final product to check whether the software meets the business needs. In simple words the test execution which we do in our day to day life are actually the validation activity which includes: smoke testing, functional testing, regression testing, systems testing...etc.

Verification and Validation (contd.)



Verification and Validation (contd.)

Verification	Validation
Are we implementing the system right?	Are we implementing the right system?
Evaluating products of a development phase	Evaluating products at the closing of the development process
The objective is making sure the product is as per the requirements and design specifications	The objective is making sure that the product meets user's requirements
Activities included: reviews, meetings, and inspections	Activities included: black box testing, white box testing, and grey box testing
Verifies that outputs are according to inputs or not	Validates that the users accept the software or not
Items evaluated: plans, requirement specifications, design specifications, code, and test cases	Items evaluated: actual product or software under test
Manual checking of the documents and files	Checking the developed products using the documents and files

Trace Requirements

Requirements Tracing is the process of recording logical links between individual requirements and other systems elements.

You can trace a single functional requirement **backward** to its origin, such as use case, product feature or business rule.

You can also trace that functional requirement **forward** into its bits of design, code and tests that were created because of that requirement.

To do requirements traceability, the analyst must write requirements in a fine-grained fashion and give every requirement a unique and stable identifier.

Start performing traceability by linking functional requirements to individual tests that verify the correct implementation of those requirements.

Requirements Traceability Matrix Example

REQUIREMENTS TRACEABILITY MATRIX					
Project Name: Online Flight Booking Application					
Business Requirements Document BRD		Functional Requirements Document FSD			Test Case Document
Business Requirement ID#	Business Requirement / Business Use case	Functional Requirement ID#	Functional Requirement / Use Case	Priority	Test Case ID#
BR_1	Reservation Module	FR_1	One Way Ticket booking	High	TC#001 TC#002
		FR_2	Round Way Ticket		TC#003 TC#004
		FR_3	Multicity Ticket booking	High	TC#005 TC#006
BR_2	Payment Module	FR_4	By Credit Card	High	TC#007 TC#008
		FR_5	By Debit Card	High	TC#009
		FR_6	By Reward Points	Medium	TC#010 TC#011

Requirements Traceability Matrix Example (contd.)

Test Case ID#	Test Case Description
TC#001	Verify if user is able to book one way ticket
TC#002	Verify if user is able to book multiple one way tickets
TC#003	Verify if user is able to book round way ticket
TC#004	Verify if user is able to book multiple round way tickets
TC#005	Verify if user is able to book one way ticket for multiple cities
TC#006	Verify if user is able to book round way ticket for multiple cities
TC#007	Verify if usre is able to pay by Master Card
TC#008	Verify if usre is able to pay by Visa Card
TC#009	Verify if usre is able to pay by Debit Cards
TC#010	Verify if usre is able to pay Fully by Reward Points
TC#011	Verify if usre is able to pay partially by Reward Points

Forward traceability

This matrix is used to check whether the project progresses in the desired direction and for the right product.

It makes sure that each **requirement** is applied to the product and that each **requirement** is tested thoroughly.

It maps **requirements** to test cases.

Backward Traceability

Mapping test cases to **requirements** is called **Backward Traceability** Matrix.

It is used to ensure whether the current product remains on the right track.

It makes sure that we are not expanding the scope of the project by adding functionality that is not specified in the **requirements**.

Benefits of requirements traceability?

(1) Management of the solution scope

Since traceability allows each requirement to be associated with the appropriate business objectives, we can evaluate the value of each requirement.

This allows us to effectively prioritize and avoid **scope creep** (the frustrating sensation of there being never-ending requirements for this project).

Benefits of requirements traceability? (contd.)

(2) Quick evaluation of potential changes

For a given requirement, we can identify the related business objectives and other affected components.

In addition, traceability allows easy identification of requirements associated with a failed test case, which supports accelerated resolution of problems.

(3) Reduced project risk

Traceability allows for identification of critical dependencies between requirements, supporting better visibility and control of these relationships.

Benefits of requirements traceability? (contd.)

(4) Promotes consistency between requirements

For example, if we use two different terms to refer to the same entity, such as Client and Registered Customer, connecting related requirements may help us see the inconsistency more easily.

(5) Allows monitoring and control across the lifecycle of requirements

The traceability matrix can be used to help manage which requirements are validated, which are pending, and which have been rejected.

It also helps in identifying which requirements correspond to a specific release.

Sign Off

Sign-offs are an indication that stakeholders agree with and approve the **requirements** that have been elicited and documented.

